CS411 - PT1 Stage 4: Checkpoint 2 and Project Report
DataDucks / Team 092: Yi-Hsin Chang, Zoe Hanson, Chun-Wen Liou

1. **Please list out changes in the directions of your project if the final project is different from your original proposal (based on your stage 1 proposal submission).**

   The primary change was the decision to move away from live time API. We initially planned to integrate the Ticketmaster API, Amadeus API, and Google Places API directly into our production backend. However, based on feedback regarding potential rate-limiting, complexity, and stability issues associated with external APIs, we decided to rely on static and pre downloaded datasets for core functionality.

2. **Discuss what you think your application achieved or failed to achieve regarding its usefulness.**

   Achievements in Usefulness

   - Integrated Planning Workflow: The application successfully combined concert search, accommodation selection (via city-match), and itinerary creation into a single, seamless flow, eliminating the need for users to switch between separate event and travel booking sites.
   - Community and Inspiration: The implementation of the public itinerary sharing feature (using the Visibility flag in the TRIP table) and the ability to copy others' trips provided significant social value, encouraging planning and discovery.
   - Near-Term Focus: By limiting the concert search to events within a short, defined window (five days), the platform effectively serves its niche as a tool for quick or spontaneous travel planning.

   Failures in Usefulness (Compare to the original proposal)

   - Nearby Hotel Recommendation: The proposal's mentioning automatically recommends nearby hotels of each venue was not achieved. We simplified this to a city match function (v.city = h.city). While this still provides relevant hotels in the same metropolitan area, it lacks the true spatial awareness needed to guarantee proximity to the venue, thereby reducing the convenience factor.

3. **Discuss if you changed the schema or source of the data for your application**
   The source of our data moved from live APIs to pre-download static datasets derived from the Ticketmaster API and Kaggle.

Component	Stage 1	Final
Concerts	Ticketmaster API (live)	Ticketmaster data download(static)
Hotels	Amadeus API + scraped reviews	Kaggle hotel dataset (static)
Geolocation	Google Maps API	Removed
Reviews	NLP summaries	Included in Kagle hotel dataset

Static datasets allowed faster debugging, ensured reproducibility, and removed dependency risk.

4. **Discuss what you change to your ER diagram and/or your table implementations. What are some differences between the original design and the final design? Why? What do you think is a more suitable design?**

   After the Stage 2 revision, we did not make additional changes to our ER diagram or table implementations, so the final design is identical to the revised Stage 2 version. The main differences between the original and final designs come from the revisions we made in response to earlier feedback. First, in the original relational schema, the SAVE table used (UserId, TripId) as a composite primary key but did not declare them as foreign keys. In the final design, we explicitly added foreign key constraints from SAVE.UserId to USER.UserId and from SAVE.TripId to TRIP.TripId. This change tightens referential integrity by ensuring that every saved record must correspond to an existing user and an existing trip, which is more consistent with the semantics of the SAVE relationship.

   Second, we revised how location information is modeled for the HOTEL and VENUE entities. In the original design, we included latitude and longitude, which implied a functional dependency from (Latitude, Longitude) to (State, City) and meant the schema was not fully in Third Normal Form. In the final design, we replaced latitude and longitude with Address and Zipcode, and kept City and State as separate attributes. This revision removes the problematic functional dependency, makes the schema closer to 3NF, and provides a more intuitive and application-friendly representation of location. For our use case—displaying hotels and venues and potentially grouping them by area—

storing address and zipcode is more practical than storing coordinates. Overall, we consider the final design more suitable because it enforces stronger referential integrity and improves normalization while still supporting the application's real needs.

5. **Discuss what functionalities you added or removed. Why?**

   Added (not in original proposal)

- Trip statistics summary card
- Stored procedure for analytics
- Automatic visibility privacy system controlled by triggers

  Removed (was in original proposal)

- Distance-based feasibility engine
- Map visualization
- Real-time API integrations

  The added features support SQL course requirements in the stage 4 and make the web page workflow more reasonable.
  The removed features required API engineering and it is too complex and time consuming for us.

6. **Explain how you think your advanced database programs complement your application.**
   Our application integrates three major types of advanced database programs, stored procedures, triggers, and transactions.

   Each of these programs supports different aspects of our system's reliability, consistency, performance, and user experience.

   Transaction-Based Stored Procedure:
   `sp_copy_public_trip_to_user`

   The transaction-based stored procedure strengthens the application by making the "Save to My Trips" feature reliable and consistent. By handling all validations and the final insertion within a single transaction, it ensures that a trip is copied only when every requirement is met, preventing duplicate events and eliminating the risk of partial or incorrect records. This design guarantees that users always experience predictable outcomes, even when many people copy the same trip simultaneously. Centralizing all business logic in the database also reduces application-side errors and ensures data

integrity across concurrent operations. As a result, the feature becomes more robust, trustworthy, and aligned with real-world multi-user application needs.

Analytical Stored Procedure:

`sp_get_user_trip_stats`

The stored procedure provides aggregated analytical data for a user's dashboard. Instead of relying on the application layer to compute metrics for the total number of trips, the number of public trips, or the count of distinct cities visited, the procedure performs these operations directly in the database. Executing analytics at the database level reduces frontend workload, improves performance as the dataset grows, and delivers immediate, accurate insights to users accessing the "My Trip Stats" section of the application.

Post Quality Triggers:

`trg_trip_visibility_before_insert &`
`trg_trip_visibility_before_update`

The application also incorporates data quality enforcement through two triggers that automatically adjust trip visibility when necessary. Each time a trip is inserted or updated, the triggers verify whether the record contains essential information such as a selected hotel and a sufficiently detailed user note. If these conditions are not met, the visibility is automatically set to Private, ensuring that incomplete or low-quality trips or draft trips do not appear on the community page. Because these rules are enforced within the database rather than the application layer, the system consistently maintains data quality even if frontend validation changes or external scripts interact with the database. This approach protects user privacy, improves the quality of publicly available itineraries, and ensures that the community section remains meaningful and reliable.

7. **Each team member should describe one technical challenge that the team encountered. This should be sufficiently detailed such that another future team could use this as helpful advice if they were to start a similar project or where to maintain your project.**

Chun-Wen Liou

One technical challenge I faced was getting the backend to work smoothly with the frontend, especially stored procedures, triggers, and transactional logic. At first, I focused on making sure the backend functions were correct, but I quickly realized that the frontend had no clear way of interpreting what the backend was doing. For example, the "Save to My Trips" feature involves multiple validation steps inside a transaction, but the frontend originally treated it as a simple insert, which led to confusing or inconsistent UI behavior when something failed. I had to restructure the backend responses, define

meaningful return codes, and coordinate closely with the frontend to test how each scenario should be handled. Once we aligned our expectations, the application became much more stable. My main takeaway is that backend work isn't just about writing correct SQL, it's about making sure the frontend can understand and interact with it properly.

Yi-Hsin Chang

My technical challenge is that I could not successfully connect to the cloud-database from vcp at first even with the information needed: the local host/port, database, and password, but then I tried to reach it from my local device and it sorts the issue using the needed information.

Zoe Hanson

One technical challenge I encountered involved building a front end that stayed consistent with our database constraints, especially after we began adding triggers and stored procedures. Early in development, the interface allowed users to add concerts and create trips in ways that did not match the logic enforced in the backend. For example, the TRIP table uses triggers that automatically set a trip to private if a user does not provide a hotel or a meaningful note. The initial front end design did not communicate this rule clearly, which caused confusion because a user could add a trip and then not see it appear on the community page.

To resolve this, I redesigned the flow of the concert detail page to align with the new database rules. I added a separate button for adding a concert without a hotel, and I updated the confirmation messages so that users understand when a trip will be private. I also adjusted how the front end passes optional fields to the backend so that the API receives information in the expected format. This helped prevent errors related to missing or incorrectly typed values.

The main lesson I learned is that front end logic cannot be developed in isolation. Once the backend begins using triggers and stored procedures, the interface must be updated to reflect the same rules or users will experience inconsistent behavior. Any future teams should communicate frequently across roles and confirm that every database constraint is represented somewhere in the user interface. This alignment makes the application easier to use and reduces bugs that arise from mismatched assumptions.

8. **Are there other things that changed comparing the final application with the original proposal?**

The most notable difference is the simplification of the recommendation engine. The original proposal envisioned a sophisticated, location-aware recommender. The final application shifted to a functional, database-centric model where the recommendation is merely a filter: Hotels are recommended if the HOTEL.City matches the VENUE.City associated with the event. This city-match is a practical replacement for the complex geospatial calculation. Other than that most aspects of the final application matched our initial plans.

9. **Describe future work that you think, other than the interface, that the application can improve on**
    - Real API Integration (Ticketmaster, Amadeus)
    - NLP Hotel Review Summaries
    - Map + Routing Feasibility Engine
    - Recommendation System **"**Users who attended X also visited Y**"**

10. **Describe the final division of labor and how well you managed teamwork.**

    Our team divided the work based on individual strengths while remaining flexible during later stages of the project. Yi-Hsin focused primarily on the backend database implementation, including writing SQL queries, designing stored procedures, and managing the connection to the cloud database. Chun-Wen concentrated on backend logic, API development, and integrating advanced database features such as triggers and transaction-based procedures. Zoe focused on the full front end, including page structure, styling, user interaction design, and ensuring that the interface aligned with database rules and backend responses. Although our roles were distinct, we communicated regularly to ensure that the front end, backend, and database schema stayed consistent with each other. We held short check-ins during development, shared debugging responsibilities, and tested features together before integration. The teamwork was effective because each member completed their part reliably and adapted when another part of the system needed support.